

Teknoloji Ekiplerinde Temel Liderlik ve Sorumluluk Kavramları

Ömer Faruk Yavuz

November 2025

Giriş

Özellikle teknoloji ekiplerinde, ürün ve proje geliştirme kültüründe bazı liderlik ve sorumluluk kavramları, hem günlük işleyişi hem de uzun vadeli performansı belirleyen kritik yapı taşlarıdır. Bu kavramlar yalnızca “kimin ne yaptığı” ile ilgili değildir; aynı zamanda rol tanımı, beklenti yönetimi, hesap verebilirlik, karar alma mekanizmaları ve ekip kültürünün *nasıl* şekillendiği ile ilgilidir.

Bu dokümanda aşağıdaki kavramlar detaylandırılmaktadır:

- Ownership
- Accountability
- Responsibility
- Stewardship
- Due Diligence
- Authority
- Autonomy
- Alignment
- Psychological Safety
- Servant Leadership
- Blameless Culture
- Continuous Improvement (Kaizen)
- RACI Modeli
- Alignment ve buna bağlı çerçeveler:
 - Definition of Ready (DoR)
 - Definition of Done (DoD)

- RACI Modeli
- Önleyici ve Sorumlu Liderlik Boyutları

Amaç, bu kavramları yalnızca teorik tanımlar olarak değil, teknoloji ekiplerinin gündelik pratiklerine uygulanabilir, gözlemlenebilir davranış setleri olarak netleştirmektir. Ayrıca, bu kavramları destekleyen yardımcı çerçeveler olarak Definition of Ready (DoR), Definition of Done (DoD) ve önleyici/sorumlu liderlik boyutları da ele alınacaktır.

Ownership

Tanım

Ownership, bir işi, problemi veya alanı *sahiplenme* zihniyetidir. Basitçe, “Bu benim işim değil.” demek yerine, “Bu problemin çözülmesinde benim de payım var.” diyebilmektir. Ownership bir unvan değil, bir **tavırdır**.

Zihniyet ve Felsefe

Ownership, görev tanımının sınırlarında durmayan, problemin gerçekten çözülmesini hedefleyen bir yaklaşımı ifade eder. Bu, genellikle şu cümle ile özetlenir:

“Bu problem çözülene kadar buradayım.”

Bu zihniyet, sadece kendi task’ına odaklanmayı değil, uçtan uca değer zincirini gözetmeyi gerektirir:

- Sadece PR açmak değil, prod ortamda çalıştığını doğrulamak.
- Sadece “ticket kapatmak” değil, kullanıcı deneyimini gerçekten iyileştirmek.
- Engel çıktığında, suçlu aramak yerine çözüm üretmek.

Davranış Örnekleri

- Engelle karşılaştığında:
 - “*Backend bekliyorum, yapamıyorum.*” demek yerine,
 - Gerekiyorsa ilgili kişilerle iletişime geçmek, alternatif çözüm önermek, riski görünür kılmak.
- Bug raporu geldiğinde, “Bu benim modulüm değil.” diyerek geri itmek yerine, sistemin bütününe bakıp en azından doğru kişiye yönlendirmek.
- Dokümansız bölgeyle karşılaşınca şikâyet etmek yerine, minimum da olsa not alıp ekibe kazandırmak.

Sık Görülen Yanlış Anlama

Ownership, *her şeyi tek başına yapmaya çalışmak* değildir. Doğru yorum:

- Yardım istemek, eskale etmek ve iş birliği yapmak hâlâ ownership kapsamındadır.
- Önemli olan, “Bu iş sahipsiz kalmasın.” diyebilen bir kişinin olmasıdır.

Accountability

Tanım

Accountability, bir işin veya sonucun nihai olarak **kimin sorumluluğunda** olduğunu ifade eder. Başarı durumunda krediyi, başarısızlık durumunda ise gerekçe yerine *hesap verebilirliği* gerektirir.

Responsibility ve Accountability Farkı

- **Responsibility:** Görevi fiilen yapan, işi yürüten taraf.
- **Accountability:** Sonuçtan nihai olarak sorumlu taraf.

Birden fazla kişi aynı işte *responsible* olabilir; ancak *accountable* olan kişi genellikle **tektir**. Bu ayrım, “Sorumluluk herkesin, dolayısıyla hiç kimsenin.” şeklindeki kaosu engeller.

Teknoloji Ekiplerinde Örnek

Bir özelliğin (feature) canlıya alınması sürecinde:

- Geliştirici(ler) kod yazma, test etme açısından *responsible* olabilir.
- QA mühendisi, test süreci için *responsible* olabilir.
- Product Owner veya Tech Lead, bu özelliğin zamanında ve doğru kalitede tesliminden *accountable* olabilir.

Başarısızlık durumunda:

- Accountable kişi, “Neden olmadı?” sorusuna net cevap vermeli,
- Sistemdeki eksikleri görünür hâle getirmeli ve iyileştirme planı önermelidir.

Accountability’nin Kime Ait Olduğunu Belirlemek

Accountability, teorik bir kavram olmanın ötesinde, pratikte her durumda “Bu işin sonucundan *nihai olarak* kim sorumludur?” sorusuna verilen tekil cevaptır. Kimin accountable olduğuna genel kanı değil, organizasyonun yapısı ve bağlam karar verir; ancak teknoloji ekiplerinde sık görülen bazı örüntüler (pattern) vardır.

Feature Geliştirme Sürecinde Accountability

Örnek senaryo:

Bir ürün için yeni bir özellik (*feature*) geliştirilmektedir.

Bu durumda tipik rol dağılımı şöyle olabilir:

- **Responsible (yapanlar):**
 - Backend geliştiricisi,

- Frontend geliştiricisi,
- QA mühendisi.
- **Accountable (sonuçtan nihai sorumlu):**
 - Ürün odaklı yapılarda çoğunlukla Product Owner veya Product Manager,
 - Daha teknik odaklı yapılarda Tech Lead veya Team Lead.

Ürün odaklı kültürde Product Manager / Product Owner:

- Özelliğin doğru problemi çözüp çözmediği,
- Beklenen iş değerini üretip üretmediği,
- Belirlenen takvimde teslim edilip edilmediği açısından hesap verebilir konumdadır.

Teknik odaklı kültürde Tech Lead / Team Lead:

- Çözümün mimari uygunluğu,
- Performans, güvenlik ve kod kalitesi,
- Takımın teslim kabiliyeti açısından accountability üstlenir.

Production Incident Durumunda Accountability

Senaryo:

Canlı (production) ortamda bir kesinti yaşanmış ve sistem belirli bir süre boyunca hata (örneğin HTTP 500) üretmiştir.

Bu durumda tipik rol dağılımı:

- **Responsible:**
 - Log ve metrikleri inceleyen geliştiriciler,
 - Hotfix veya rollback uygulayan geliştiriciler,
 - DevOps / SRE ekibi.
- **Accountable:**
 - O incident için atanmış *Incident Commander (IC)*,
 - veya ilgili Team Lead / Engineering Manager.

Accountable olan kişi, aşağıdaki sorulara net cevap vermekle yükümlüdür:

- Olay neden meydana geldi?
- Nasıl ve ne kadar sürede fark edildi? (monitoring ve alert mekanizmaları)
- Neden daha erken yakalanmadı? (test süreçleri, kalite kontrol adımları)
- Tekrar etmemesi için hangi iyileştirme adımları planlandı ve uygulandı?

Sprint Hedefleri ve Takım Çıktısı İçin Accountability

Bir “squad” ya da scrum takımı düşünüldüğünde, tipik bir bileşimde:

- Birden fazla geliştirici,
- QA mühendisi,
- Ürün tarafında Product Owner veya Product Manager bulunur.

Bu bağlamda:

- **Responsible:** Takımdaki bireyler, kendi görevlerini (task, story) yerine getirmekten sorumludur.
- **Accountable:**
 - Takımın teknik çıktısı ve teslim kabiliyeti açısından çoğunlukla Team Lead veya Engineering Manager,
 - İş değeri ve doğru öncelik seçimi açısından ise Product Owner / Product Manager accountable kabul edilir.

Bu ikili yapı, teknik sonuç ile iş sonucunu birbirinden ayırarak daha net hesap verebilirlik sağlar:

- Teknik kalite ve sürdürülebilirlik → teknik liderlik,
- Ürün değeri ve roadmap uyumu → ürün liderliği tarafından sahiplenilir.

PR (Pull Request) Sürecinde Accountability

Kod review ve PR süreçlerinde de benzer bir ayırım yapılabilir:

- **Responsible:**
 - PR’ı açan geliştirici (kodun implementasyonundan sorumlu),
 - Review yapan geliştiriciler (kod kalitesine ve standarda uygunluğa katkı sağlayan).
- **Accountable:**
 - İlgili repository veya modül için belirlenmiş Code Owner,
 - veya Tech Lead / Team Lead.

Bu kişi veya rol:

- Kod kalitesinin genel standardını tanımlar,
- PR süreçlerine ilişkin beklentileri (örneğin incremental PR kabul edilip edilmediğini) netleştirir,
- Review sürecinin sağlıklı işlemlerinden ve ekibe bunun doğru anlatılmasından sorumludur.

Dolayısıyla, PR sürecindeki teknik detaylardan birden fazla kişi *responsible* olabilirken, review kültürü, beklenti yönetimi ve standartların netliğinden nihai olarak **accountable** olan taraf, çoğunlukla liderlik veya code ownership rolüdür.

WIP PR ve Incremental Çalışma Kültürü

Work-in-progress (WIP) PR açmak, modern yazılım ekiplerinde:

- Erken geri bildirim almak,
- Bloajları önceden tespit etmek,
- Bilişsel yükü (*cognitive load*) azaltmak,
- İteratif teslimat (*incremental delivery*) sağlamak

için kullanılan **sağlıklı ve yaygın** bir pratiktir. Pek çok yerde “*open early, iterate fast*” tarzı rehberliklerde özendirilen bir çalışma biçimidir. Bununla birlikte, her ekip bu yaklaşımı aynı şekilde yorumlamaz. PR kültürü kabaca iki uca ayrılabilir:

	Iterative & Agile Kültür	High-rigor / Final PR Kültürü
PR Algısı	Süreç içinde gelişen, iteratif bir artefakt	Bitmiş, production’a yakın son çıktı
Amaç	Erken geri bildirim, yaklaşımı doğrulama	Son kalite kontrol, son halin onayı
Refactor / Test	Sonraki adımlara bölünebilir, incremental	Çoğunlukla PR içinde tamamlanmış olmalı
WIP PR	Normal, hatta teşvik edilen pratik	Çoğu zaman hoş karşılanmaz, kafa karıştırıcı bulunur

Tablo 1: PR kültüründe iki yaygın uç yaklaşım

Bu iki kültür birbirine açıkça anlatılmadığında, niyet ve algı birbirine karışır: Bir taraf için “*erken görünürlük ve iş birliği*” olan şey, diğer taraf için “*bitmemiş ve eksik iş*” gibi görünebilir. Bu nedenle, aynı teknik davranış (örneğin incremental PR açmak) bir organizasyonda “best practice” sayılırken, başka bir organizasyonda yanlış anlaşılabilir. Bu durumda problem davranışın kendisinden çok, **beklentinin önceden yazılı ve sözlü olarak netleştirilmemiş, bir iletişim formatında konuşulmamış olmasıdır**.

Accountability’yi Pratikte Tespit Etmek

Herhangi bir durumda accountability’nin kimde olduğunu pratik olarak anlamak için şu soru yol gösterici olabilir:

“Bu iş başarısız olursa, üst yönetim seviyesinde (örneğin CEO’ya) çıkıp açıklama yapmak zorunda kalan tek kişi kim olurdu?”

Bu soru genellikle aşağıdaki rollere işaret eder:

- Bir özellik ya da ürün için: Product Manager / Product Owner,
- Bir takımın teknik çıktısı için: Team Lead / Engineering Manager,

- Mimari kararlar için: Tech Lead / Principal Engineer,
- Proje bütçesi ve zaman planı için: Project Manager.

Eğer bir organizasyonda belirli bir konu için “accountable” olan kişi net değilse, bu durum genellikle rol belirsizliğine ve sorun anında “herkesin birbirine baktığı” bir kasa işaret eder. Bu nedenle accountability’nin bilinçli ve açık şekilde tanımlanması, hem performans hem de ekip içi güven açısından kritiktir.

Responsibility

Tanım

Responsibility, belirli bir görevi veya işi fiilen üstlenen ve yürüten kişi veya kişileri ifade eder. Yani “*Kim yapıyor?*” sorusunun cevabıdır.

Örnekler

- Bir endpoint’in implementasyonunu yazmak.
- PR açmak, kodu review’a hazır hâle getirmek.
- Test senaryolarını yazmak ve otomatik testleri çalıştırmak.
- Deployment script’lerini güncellemek.

Paylaşılabilir Olması

Responsibility genellikle **paylaşılabilir**:

- Aynı feature üzerinde birden fazla geliştirici çalışabilir.
- Hem backend hem frontend tarafında farklı kişiler responsible olabilir.

Buna karşılık accountability çoğu zaman **tek bir kişide** toplanır; bu da rol netliğini artırır.

Stewardship

Tanım

Stewardship, eldeki kaynakları (ekip, zaman, kod tabanı, bütçe vb.) bir *emanet* gibi görmek ve yalnızca kısa vadeli sonuçlara değil, **sürdürülebilirliğe** odaklanmak anlamına gelir. Sadece “iş yapmak” değil, sistemi geleceğe de sağlıklı şekilde taşıma sorumluluğudur.

Örnek Davranışlar

- Hızlı çözüm ile sürdürülebilir çözüm arasında karar verirken, sadece bugünü değil, 6–12 aylık etkiyi düşünmek.
- Teknik borcu sürekli ertelemek yerine, gerçekçi azaltma planları önermek.
- Takım arkadaşlarının tükenmişliğini (burnout) ciddiye alıp iş yükü ve süreçleri buna göre dengelemeye çalışmak.

Teknik Borç Bağlamı

Stewardship, “En hızlı patch’i atalım, gerisini sonra düşünürüz.” yaklaşımına mesafelidir. Bunun yerine:

- Kısa vadeli workaround’lar gerekli olduğunda bile,
- Orta vadeli refactoring planının görünür olması,
- Kod kalitesinin ve mimarinin zamanla çürümesine izin verilmemesi, gerektiğini vurgular.

Due Diligence

Tanım

Due diligence, kabaca “**gerekli özeni göstermek**” anlamına gelir. Bir işi yaparken, makul bir profesyonelden beklenen dikkat ve titizliği göstermektir. “Yapmış olmak için yapmak” ile arasına çizgi çeker.

Örnekler

- Kritik bir modülde yalnızca happy path’i değil, edge case’leri de düşünmek ve test etmek.
- PR açmadan önce statik analiz, unit test ve temel manuel kontrolleri çalıştırmak.
- Deployment öncesi rollback planı, monitoring ve alarm mekanizmalarını gözden geçirmek.
- Kullanıcıya dönük bir değişiklikte, dokümantasyon veya changelog bırakmak.

Neden Önemli?

Due diligence, hem teknik riskleri hem de itibar risklerini azaltır. “Ben yaptım, gerisi şans.” yaklaşımından ziyade:

“Makul bir mühendisin yapması gerekenleri yaptım mı?” sorusuna içtenlikle “Evet.” diyebilmektir.

Kolektif Sorumluluk Boyutu

Due diligence, yalnızca tek bir kişinin omzuna bırakılabilecek bir yük değildir. Bireysel özen kritik olmakla birlikte, gerekli özenin gerçekten sürdürülebilir olabilmesi için:

- **Ekip seviyesinde** net kalite standartlarının, check-list’lerin ve test pratiklerinin tanımlanmış olması,
- **Review süreçlerinin** (kod inceleme, QA, ürün onayı) bu standartları sistematik olarak desteklemesi,
- **Liderlik tarafının** (Tech Lead, EM, PO vb.) hız–kalite dengesini gerçekçi biçimde yönetmesi ve ekibi sadece hız üzerinden baskılamaması,
- **Süreç ve araçların** (CI/CD, otomatik testler, statik analiz, monitoring) kişisel iyi niyete bağımlı bırakılmaması

gerekir.

Başka bir ifadeyle, due diligence hem bireysel bir profesyonellik beklentisi, hem de ekip ve organizasyon düzeyinde tasarlanması gereken *sistematik bir sorumluluktur*. Sistem bu özeni desteklemediğinde, tek tek bireylerden mükemmeliyet beklemek gerçekçi ve adil değildir.

Authority

Tanım

Authority, karar alma gücü ve yetkisini ifade eder. Hangi konularda kimin “son sözü” söylediğini ve hangi alanlarda inisiyatif kullanabileceğini belirler.

Adalet Boyutu

Authority olmadan accountability adil değildir. Eğer bir kişiden sonuçtan sorumlu olması bekleniyorsa:

- İlgili kararlara etki edebilmesi,
- Gerekli kaynaklara erişebilmesi,
- Öncelikleri değiştirme veya eskale etme hakkına sahip olması gereklidir.

Örnek

Bir Tech Lead’in performance optimizasyonundan hesap verebilir olması bekleniyorsa:

- Sprint önceliklerine müdahil olma,
- Gerekirse ek süre veya kaynak talep etme,
- Mimari kararlar konusunda söz sahibi olma yetkisine de sahip olması gerekir.

Autonomy

Tanım

Autonomy, bir ekibin veya bireyin, sürekli mikro-yönetim (micromanagement) olmadan işini yapabilme özgürlüğüdür. Kısaca:

“Ne yapacağımız net, *nasıl* yapacağımıza biz karar verebiliriz.”

İdeal Denge: Alignment + Trust

Sağlıklı autonomy, iki bileşenin dengesine dayanır:

- **Alignment:** Hedefler, öncelikler ve başarı kriterleri net.
- **Trust:** Bu hedefe ulaşırken kullanılan yöntemler konusunda ekibe güven duyuluyor.

Autonomy, “kimse bize karışmasın” değil; *çerçevesi belli bir özgürlüktür*.

PR Review Süreçlerinde Autonomy

Pull request (PR) süreçleri, autonomy’nin pratikte en görünür olduğu alanlardan biridir. Burada amaç, hem kod kalitesi ve standartlarını korumak, hem de geliştiricilerin teknik inisiyatif kullanabilmesine alan açmaktır.

Sağlıklı bir PR sürecinde:

- Geliştirici, çözüm tasarımı ve implementasyon kararlarında belirli bir esnekliğe sahiptir.
- Review süreci, geliştiricinin üzerinde “gizli yönetici” gibi değil, **ikinci bir güvenlik bariyeri** ve **mesleki diyalog alanı** gibi konumlanır.
- Standartlar ve beklentiler (style guide, test gereklilikleri, incremental PR kültürü vb.) önceden netleştirilmiştir; review esnasında sürpriz kural keşfi yaşanmaz.

PR Review ve Autonomy Arasındaki İnce Çizgi

PR review, autonomy’yi iki uçtan da zedeleyebilir:

- **Aşırı müdahaleci yaklaşım (micromanagement):**
 - Reviewer, her küçük tercih üzerinde ısrarla yönlendirir, geliştiricinin teknik muhakemesine alan bırakmaz.
 - Her PR, “tamamen reviewer’ın kafasındaki stile uydurma” egzersizine dönüşür; ekip standardından çok kişisel zevk baskın olur.
 - Sonuç: Autonomy hissi azalır, geliştiriciler yalnızca “nasıl isteniyorsa öyle yapma” moduna geçer.
- **Aşırı bırakılmış, sahipsiz süreç:**
 - Review neredeyse formaliteye iner; yapısal geri bildirim çok azdır.

- Ortak bir kalite standardı oluşmaz; herkesin PR'ı farklı seviyede kalır.
- Sonuç: Autonomy, *standartsızlık* ile karışır; uzun vadede teknik borç artar.

Sağlıklı denge, şu çerçeve ile özetlenebilir:

“Ne yapılacağı ve minimum kalite standardı net; bu çerçeve içinde geliştiriciye teknik çözümü tasarlama özgürlüğü tanınır.”

PR Review'de Autonomy'yi Destekleyen Pratikler

Autonomy'yi korurken kaliteyi de güvence altına almak için aşağıdaki pratikler etkili olabilir:

- **Önceden tanımlanmış standartlar:**

- Kodlama standartları, test beklentileri ve dokümantasyon gereksinimleri yazılı hâle getirilir.
- Incremental PR kabul edilip edilmeyeceği, hangi durumlarda daha büyük/refactor odaklı PR beklendiği önceden netleştirilir.

- **Niyet odaklı review:**

- Reviewer, sadece *nasıl* yazıldığına değil, *neden bu çözümün seçildiğine* odaklanır.
- Alternatif önerirken, geliştiricinin gerekçesini anlamaya çalışır; “yanlış-doğru” ikiliği yerine “daha iyi/özgün seçenekler” ekseninde konuşur.
- Geri bildirim, katı bir “bunu yap / bunu yapma” (*do / don't do*) diliyle değil; “Şöyle yapsak nasıl olurdu?”, “Bu varyantın avantajı/dezavantajı ne olabilir?”

gibi, seçenek açan ve ortak keşfi teşvik eden sorularla verilir.

- **Kırmızı çizgi + esneklik ayrımı:**

- Güvenlik, performans, veri bütünlüğü gibi konularda kırmızı çizgiler net bir şekilde tanımlanır ve bu alanlarda review daha katıdır.
- Küçük refactor önerileri gibi alanlarda geliştiriciye daha fazla takdir hakkı (discretion) bırakılır.

- **Diyalog kanalı olarak review:**

- Yorumlar, emir cümlesi yerine açıklayıcı ve tartışmaya açık formda yazılır: “Böyle yapalım.” yerine, “Şöyle yapsak şu avantajları olur mu?”
- Reviewer ve geliştirici, PR'ı ortak bir öğrenme alanı olarak görür; tek yönlü değerlendirme değil, çift yönlü fikir alışverişi esastır.

Autonomy ve Accountability İlişkisi

Autonomy, her zaman accountability ile birlikte düşünölmelidir:

- Geliştiriciye çözüm tasarlama özgürlüğü veriliyorsa, ortaya çıkan teknik sonuç için de makul bir düzeyde sorumluluk taşıdığı kabul edilir.
- Buna karşılık, ekip veya liderlik tarafı; beklenti, standart ve sınırları net tanımlamadan, yalnızca “sonuçtan sorumlusun” demekle yetinemez.

Bu nedenle sağlıklı bir yapı için şu üçlü açık olmalıdır:

1. Hedef ve başarı kriterleri (*alignment*),
2. Karar alma ve çözüm seçme özgürlüğü (*autonomy*),
3. Sonuçtan kimin hesap vereceğı (*accountability*).

Bu üçü birlikte tasarlanmadığında, autonomy ya micromanagement’e ya da sahipsizliğe dönüşür; her iki durumda da ekip motivasyonu ve kalite uzun vadede zarar görür.

Alignment

Tanım

Alignment, ekibin ve paydaşların uyumlu şekilde **aynı hedefe koşup koşmadığını** ifade eder. Herkesin farklı şeyler beklediği, farklı metriklere baktığı ortamlarda alignment zayıftır.

Araçlar

Teknoloji ekiplerinde alignment sağlamak için sık kullanılan araçlar:

- OKR (Objectives and Key Results)
- Roadmap ve release planları
- Sprint hedefleri (sprint goals)
- Definition of Done (DoD) ve Definition of Ready (DoR)
- Net kabul kriterleri (acceptance criteria)

Belirtiler

- Güçlü alignment:
 - İnsanlar farklı rollerde olsalar bile, “Bu çeyrekte bizim için en kritik şey nedir?” sorusuna benzer yanıtlar verir.
- Zayıf alignment:
 - Ekip üyeleri birbirinden habersiz farklı önceliklere koşar,
 - Sık sık “Ama ben öyle anlamamıştım.” cümlesi duyulur.

Definition of Ready (DoR)

Definition of Ready (DoR), bir işin veya user story’nin *başlamaya hazır* olup olmadığını belirleyen çerçevedir. Amaç, geliştirme süreci başlamadan önce story’nin yeterince net, anlaşılır ve uygulanabilir olduğundan emin olmaktır. Başka bir deyişle DoR, şu soruya cevap verir:

“Bu işe başladığımızda, büyük sürprizler ve temel belirsizlikler yaşayacak mısınız?”

Aşağıda örnek bir user story için DoR kriterleri listelenmiştir.

Örnek User Story

User Story: “Bir kullanıcı olarak, profil sayfamdan bildirim tercihlerini güncelleyebilmek istiyorum ki, hangi e-posta ve push bildirimlerini alacağımı kendim yönetebileyim.”

Örnek DoR Kriterleri

Aşağıdaki kriterler sağlanmadan bu user story sprint’e alınmaz; yani “başlamaya hazır” kabul edilmez.

#	DoR Kriteri
1	Story’nin amacı ve kapsamı net; hangi bildirim türlerinin yönetileceği (e-posta, push, SMS vb.) tanımlanmış.
2	Acceptance kriterleri yazılmış ve ekip tarafından anlaşılmış.
3	Gerekli UI tasarımı veya wireframe, tasarım aracında hazır ve linki backlog item’ına eklenmiş.
4	API tarafında ihtiyaç duyulacak endpoint’ler (varsa) tanımlanmış veya ilgili backend ekibi ile netleştirilmiş.
5	Bağımlılıklar (başka takım, 3rd party servis, feature flag vb.) listelenmiş ve riskler görünür hâle getirilmiş.
6	Story, tek sprint içinde tamamlanabilecek boyuta bölünmüş; gerekirse daha küçük parçalara ayrılmış.
7	Test edilebilirlik (hangi durumların test edileceği) kabaca tanımlı.

Tablo 2: Örnek Definition of Ready (DoR) kriterleri

Definition of Done (DoD)

Definition of Done (DoD), bir işin veya user story’nin gerçekten *bitti* sayılması için sağlanması gereken ortak kalite ve teslim kriterlerini tanımlar. Yalnızca “kod yazıldı” veya “develop branch’ine atıldı” olmak, DoD’ye göre “done” olmak için yeterli değildir. DoD şu soruya odaklanır:

“Bu işin gerçekten tamamlandığını, güvenle teslim edilebilir olduğunu söyleyebilir miyiz?”

Örnek DoD Kriterleri

Aşağıdaki kriterler sağlanmadan bu user story “bitti” olarak işaretlenmez:

DoR ve DoD’nin Birlikte Kullanımı

DoR ve DoD birlikte kullanıldığında:

- Planlama aşamasında belirsizlikler azalır; story’ler hazır olmayan hâllerıyla sprint’e alınmaz.
- Sprint sonunda “aslında tam bitmemiş” işlerin done kolonuna taşınması engellenir.
- Ekip içinde “başlamak için asgari netlik” ve “bitti demek için asgari kalite” kavramları standardize edilmiş olur.

#	DoD Kriteri
1	Tüm acceptance kriterleri karşılanmış ve senaryolar manual testlerle doğrulanmış.
2	İlgili backend ve frontend kodu yazılmış, unit test'ler eklenmiş ve tüm testler başarıyla geçiyor.
3	Lint / static analysis hatası bulunmuyor; proje standartlarına uygunluk kontrol edilmiş.
4	PR açılmış, en az bir ekip üyesi tarafından review edilip onaylanmış ve code review yorumları adreslenmiş.
5	Feature, hedef branch'e (örneğin main veya develop) merge edilmiş ve CI pipeline başarılı şekilde tamamlanmış.
6	Gerekli dokümantasyon (örn. ayarlar ekranına dair kısa açıklama, API contract değişikliği, changelog girdisi) güncellenmiş.
7	Gerekliyse feature flag ile çevrelenmiş; açma/kapama stratejisi ve rollback planı belirlenmiş.
8	Temel monitoring / logging noktaları eklenmiş ve production'da davranışın izlenebilmesi için asgari sinyaller tanımlanmış.

Tablo 3: Örnek Definition of Done (DoD) kriterleri

Psychological Safety

Tanım

Psychological safety, ekip üyelerinin hata yapmaktan, soru sormaktan veya fikir paylaşmaktan dolayı cezalandırılma, alay edilme veya dışlanma korkusu yaşamadan konuşabildiği ortamı ifade eder.

Neden Kritiktir?

Yüksek performanslı ekiplerin ortak özelliklerinden biri, önce bir **öğrenme kültürü** inşa etmeleridir. Üretkenlik, çoğu zaman bu zeminin üzerine kurulabilir:

- İnsanlar hata yapma korkusuyla susuyorsa, problemler erken aşamada görünmez kalır.
- “Ben mi yanlış anladım?” diyemeyen bir ekip, kritik yanlışları geç fark eder.

Davranış Örnekleri

- Senior bir mühendisin, kendi hatasını ekibin önünde açıkça paylaşabilmesi.
- Retrospective toplantılarında kişi suçlamak yerine, sistem ve süreç konuşulması.
- Yeni başlayan birinin “Anlamadım, bir daha anlatır mısın?” sorusuna sabırla açıklama yapılması.

Servant Leadership

Tanım

Servant leadership, liderin gücünü insanlara hükmetmekten değil, onların önündeki engelleri kaldırmaktan alan liderlik anlayışıdır. Otorite kaynağı, unvandan çok **hizmet etme** motivasyonudur.

Temel Slogan

“Ben emredeyim.” yerine, “Ben destek olayım.” yaklaşımı.

Pratikte Ne Demektir?

- Ekip üyelerinin önündeki süreçsel, teknik veya organizasyonel engelleri aktif şekilde tespit edip kaldırmak.
- Sadece “ne yapılacağını” söylemek değil, “neden yapıldığını” birlikte netleştirmek.
- Kredi dağıtırken ekibi öne çıkarmak, sorumluluk alırken kendini öne koymak.

Blameless Culture

Tanım

Blameless culture, hatanın önce **kişide** değil, **sistemde** aranmasını esas alan kültürdür. Amaç, suçlu bulmak değil, tekrar etmemesini sağlayacak öğrenmeyi maksimuma çıkarmaktır.

Temel Felsefe

“Who did it?” yerine, “How do we prevent it next time?”

Örnek Uygulamalar

- Prod incident sonrasında postmortem yapılırken, isimler üzerinden değil, süreçler üzerinden konuşmak.
- “Hata kimin?” sorusundan çok, “Hangi sinyal eksikti?”, “Hangi check list yoktu?” gibi sistem sorularını sormak.
- Aynı hatanın tekrar etmemesi için aksiyon maddelerini takip etmek ve görünür tutmak.

Continuous Improvement (Kaizen)

Tanım

Continuous improvement veya Kaizen, büyük devrimsel değişikliklerden ziyade, küçük ama **sürekli** iyileştirmelere odaklanan bir yaklaşımdır. Bu iyileştirme:

- İnsan,
- Süreç,
- Teknoloji

alanlarının tamamını kapsar.

Teknoloji Ekiplerinde Örnekler

- Her sprint retrospective’inde, en az bir küçük süreç iyileştirmesini denemek.
- Tooling tarafında, manuel ve tekrarlı işleri yavaş yavaş otomasyona taşımak.
- Kod review pratiğini, ekip içi geri bildirimlerle zaman içinde daha net ve verimli hâle getirmek.

Zihniyet

Kaizen, “Mükemmele ulaşalım.” demekten çok,

“Dünkü hâlimizden neyi *bir tık* daha iyi yapabiliriz?”

sorusunu düzenli olarak sormaktır.

RACI Modeli

Tanım

RACI modeli, bir iş veya inisiyatif özelinde rol ve sorumlulukları netleştiren klasik bir çerçevedir. Dört temel rol içerir:

- **Responsible (Yapan)**
- **Accountable (Hesap veren / Sonuçtan sorumlu)**
- **Consulted (Danışılan)**
- **Informed (Bilgilendirilen)**

Rollerin Açıklaması

Responsible: İş fiilen yapan kişi veya kişiler. Geliştirme, test, araştırma gibi yürütme faaliyetlerinden sorumludur.

Accountable: Sonuçtan nihai olarak sorumlu olan kişidir; çoğu zaman tektir. Bu kişi, kararların arkasında durur ve işin sahipliğini üstlenir. Sağlıklı bir yorumda, accountability baskı anlarında suçu ekibin alt seviyelerine “devretme” hakkı vermez; aksine, üst yönetim ile ekip arasında bir *kalkan* görevi görmesini gerektirir. Üst tarafta karşı sonuçtan hesap verirken, aşağıya karşı kişi odaklı suçlama yerine sistem ve süreç odaklı iyileştirme sorumluluğunu taşır.

Consulted: Karar verilmeden önce görüşü alınan, bilgi veya uzmanlık sağlayan paydaşlardır. İletişim genellikle çift yönlüdür.

Informed: Süreç ve sonuçlardan haberdar edilmesi gereken, ancak karar sürecine doğrudan dahil olmayan paydaşlardır. İletişim genellikle tek yönlüdür.

Amaç

RACI'nin temel amacı:

- “Kim ne yapıyor?” kaosunu azaltmak,
- Aynı iş için birden fazla *accountable* kişi olmasını engellemek,
- Yanlış beklentilerden kaynaklanan çatışmaları azaltmaktır.

Önleyici ve Sorumlu Liderlik Boyutları

Teknik liderlik ve ekip yönetimi, yalnızca kod kalitesi veya teslim süreleriyle ölçülemez. Bir liderin (ve deneyimli geliştiricinin [senior ve üzeri title'lar için]) rolü; insanları, süreçleri ve riskleri *önceden* düşünerek tasarlamayı da içerir. Bu bağlamda aşağıdaki boyutlar kritik önemdedir: hazırlıklı olmak, öngörü geliştirmek, önyargıları yönetmek, her seviyeden kişiyi dinlemek, iletişimi sadece kriz anlarına sıkıştırmamak ve sağlıklı rotasyon mekanizmaları kurmak.

Hazırlıklı Olmak

Bir işi veya ekip üyesini (stajyer dâhil) ekibe dahil etmek, aslında küçük ölçekli bir “organizasyonel taahhüt”tür. Lider veya deneyimli geliştirici, bu adımı atmadan önce şu soruları kendine sormalıdır:

- Bu kişi veya iş için hangi **beklentileri** netleştirdim?
- Gerekli **onboarding materyali**, dokümantasyon ve mentorluk kapasitesi hazır mı?
- Bu kişiye gerçekten **zaman ayırabilecek miyim**, yoksa görünürde sahiplenip pratikte yalnız mı bırakacağım?
- Bu işi ekibe almadan önce, var olan iş yükü ve önceliklere etkisini düşündüm mü?

Hazırlıksız alınan işler ve insanlar, çoğu zaman sonradan “*uyumsuzluk*” veya “*performans problemi*” etiketiyle damgalanır; oysa kökteki problem, **öncesinde yapılmayan planlama ve netliktir**.

Öngörü Geliştirmek ve Riskleri Önceden Düşünmek

Liderlik ve kıdemli geliştiricilik, yalnızca olan bitene tepki vermek değil, olası senaryoları *önceden* okumaktır. Örneğin:

- Yeni bir kişiyi ekibe alırken: İlk 3 ayda ne tür zorluklar yaşayabilir? Hangi alanlarda desteğe ihtiyaç duyacak?
- Kritik bir projeyi üstlenirken: Hangi bağımlılıklar riskli? Hangi dönemde ekip üzerinde baskı artacak?
- Süreç değişikliği yaparken: Hangi kişiler bu değişime uyumda zorlanabilir?

Bu öngörü, *önyargı* üretmek için değil, **destek planı** oluşturmak için kullanılmalıdır. “Zaten yapamaz.” demek değil, “İlk 2 ay şuralarda zorlanabilir, o yüzden şu desteği hazırlamalıyım.” demektir.

Önyargısız Yaklaşım ve Etiketleme Riski

Liderler ve deneyimli geliştiriciler, farkında olmadan şu tür etiketlere kayabilir:

“Bu kişi yavaş çalışıyor.”, “Bu kişi savunmacı.”, “Bu kişi kültüre uymuyor.”

Bu tür etiketler:

- Çoğu zaman sınırlı gözleme dayanır,
- Kişinin değişim ve gelişim potansiyelini gölgeler,
- Geri bildirim ve destek mekanizmalarını zayıflatır.

Önyargısız yaklaşım, *kişiyi* damgalamak yerine, *davranışı ve bağlamı* anlamaya çalışmayı gerektirir:

- “Bu davranış neden ortaya çıkmış olabilir?”
- “Bu kişinin hangi koşullarda daha iyi performans göstermesi mümkün?”
- “Benim veya sistemin tasarımında bu davranışı tetikleyen ne var?”

Her Seviyeden Kişiyi Dinlemek

Sağlıklı ekiplerde, **seviye ve unvan**, bir kişinin dinlenip dinlenmemesi için kriter değildir. Stajyerler, yeni başlayanlar veya junior geliştiriciler:

- Sisteme taze bir gözle bakar,
- “Biz her zaman böyle yapıyoruz.” denilen alanlarda yeni sorular sorar,
- Kullanıcıya daha yakın sezgiler getirebilir.

Liderin görevi:

- Sadece senior sesleri değil, **tüm seviyeleri** aktif olarak dinlemek,
- Güvenli bir ortamda, stajyer dahil herkesin “Bu kısmı anlamadım.” veya “Burada bir problem görüyorum.” diyebilmesini sağlamaktır.

Bu, hem *psychological safety*’yi besler, hem de kör noktaların daha erken görünür olmasını sağlar. Yeni katılan ekip üyeleri, “taze göz” olarak, sistemin dışarıdan nasıl görüldüğüne dair değerli eleştiriler ve sezgiler getirir; çoğu zaman uzun süredir içeride olanların artık “normal”leştirdiği problemleri işaret eder. Bu noktada önemli bir ayrım vardır: Yeni gelenlerin getirdiği bu taze eleştirel bakışı **sistematik olarak dinlemek** ve süreçlere yansıtmak sağlıklı bir pratiktir; ancak bunu gerekçe gösterip çalışan sirkülasyonunu bilinçli olarak yüksek tutmak veya “kolay harcanabilir insan” kültürü yaratmak, kısa vadede bazı verimlilik sinyalleri üretse bile, uzun vadede hem *psychological safety*’yi hem de kurumsal hafızayı zedeler. Gerçek eleştiri, insanların kendini güvende hissettiği, hatanın ve farklı görüşün “kovulma riski” ile otomatik ilişkilendirilmediği ortamlarda sürdürülebilir hâle gelir.

İletişimin Sadece Zor Zamanlarda Değil, Her Zaman Olması

Birçok ekipte iletişim, sadece kriz anlarında veya performans sorunu algılandığında devreye girer. Bu, hem çalışan hem lider açısından *yüksek stresli ve geç* bir temas biçimidir. Sağlıklı modelde:

- Düzenli 1:1 görüşmeler,
- Sprint sonu hızlı check-in'ler,
- Beklenti ve önceliklerin periyodik olarak üzerinden geçilmesi,
- “İyi giden şeylerin” de konuşulduğu pozitif geri bildirimler

ile iletişim, **sürekli ve öngörülebilir** bir ritme oturur. Böylece:

- Sorunlar büyümeden önce yakalanır,
- İnsanlar sadece kötü haber alacakları zaman değil, her zaman liderlerine gidebileceklerini hisseder,
- Ani ve hazırlıksız kararların sayısı azalır.

Rotasyonun Önemi

Rotasyon, insanların ve ekiplerin farklı alanlarda deneyim kazanması için bilinçli olarak tasarlanan yer değiştirme hareketidir. Bu, aşağıdaki açılardan kritik bir araçtır:

- **Birey için:**
 - Farklı domain ve teknoloji stack'leri deneyimleme fırsatı,
 - Tek bir rol veya modüle sıkışıp kalmama,
 - Güçlü ve zayıf yönlerini farklı bağlamlarda keşfetme.
- **Ekip ve organizasyon için:**
 - Bilginin tek kişide kilitlenmesini (*bus factor*) azaltma,
 - Farklı ekipler arasında empati ve anlayış geliştirme,
 - Süreç ve pratiklerin *çapraz tozlaşma* yoluyla iyileşmesini sağlama.

Rotasyon, *ceza* veya *kaçış* aracı değil, bilinçli bir gelişim ve risk yönetimi enstrümanı olarak kurgulanmalıdır. Liderlik, rotasyonu:

- İnsanların ilgi alanları ve potansiyelleri,
- Organizasyonun uzun vadeli ihtiyaçları,
- Ekipler arası denge

ile birlikte değerlendirerek planlamalıdır.

“Önceden düşünmek, önyargısız dinlemek, sürekli iletişim kurmak ve rotasyonla alan açmak”: Bunlar bir araya geldiğinde, hem bireyler hem ekipler için daha adil, daha öğrenen ve daha dayanıklı bir çalışma ortamı oluşur.

İşletme Körlüğü, Önyargı ve Emek Kültürü

Uzun süre aynı sistemin içinde kalan ekipler için, belirli problemlerin “normalleşmesi” kaçınılmazdır. Bu duruma çoğu zaman *işletme körlüğü* denir: Dışarıdan bakıldığında apaçık görülen verimsizlikler, riskler veya anlamsız rutinler, içeridekiler için gündelik hayatın sıradan bir parçasına dönüşür.

Yeni gelen ekip üyeleri, bu işletme körlüğünü kırmak için değerli bir *ayna* işlevi görür. Şu tür cümleler, aslında sistemin kendisi hakkında önemli sinyaller taşır:

- “Neden bunu böyle yapıyoruz?”
- “Burada niye bu kadar manuel adım var?”
- “Bu kuralın hâlâ geçerli olması gerekiyor mu?”

Bu soruların refleksif olarak şu tür yanıtlarla bastırılması ise *öğrenen organizasyon* olmanın önünde ciddi bir engeldir:

- “O iş öyle olmaz.”
- “Biz hep böyle yaptık.”
- “Bunlar boş işler, gerçek hayatta yürümez.”

Bu “*bu olmaz*” *önyargısı*, sadece yeni fikirleri değil, aynı zamanda insanların katkı yapma motivasyonunu da törpüler. Zamanla:

- Yeni gelenler soru sormamayı öğrenir,
- Deneyimli kişiler eleştirel düşünmeyi bırakır,
- Organizasyon, kendini içeriden güncelleyemez hale gelir.

Emek Kültürü: Sadece Sonuçları Değil, Çabayı da Görmek

Sağlıklı bir ekip kültürü, yalnızca “çıkan sonuca” değil, aynı zamanda **harcanan emeğe** de saygı duyar. Bir öneri hayata geçmese bile:

- Fikir üretme cesareti,
- Alternatifleri düşünme çabası,
- Sistemi iyileştirme niyeti

teşekkür ve takdiri hak eder.

Emek kültürünün olmadığı ortamlarda:

- İnsanlar risk almaktan ve yeni fikir getirmekten çekinir,
- Yalnızca “emin oldukları, zaten kabul edilen işleri” yapmaya yönelir,
- Uzun vadede yenilikçilik ve sürekli iyileştirme kapasitesi zayıflar.

Basit bir “*Bunu denediğin için teşekkür ederim.*” cümlesi bile, psikolojik güvenlik ve sahiplenme hissi üzerinde çarpan etkisi yaratabilir. İnsanların emeğinin görünür olması, onların *yalnızca bir kaynak* değil, emek ve niyeti olan *özneler* olarak görüldüğü mesajını verir.

Liderliğin Ton Belirleyici Rolü

Ton, en üstten belirlenir. Şu alanlarda gözlenen problemler çoğu zaman liderlik seviyesindeki tercih ve davranışların yansımasıdır:

- İnsanların fikirlerinin sürekli bastırılması,
- Hataların kişi odaklı cezalandırılması,
- Yeni gelenlerin sistematik olarak dinlenmemesi,
- “Bu olmaz.” refleksinin norm hâline gelmesi.

Liderlik, hem *neye izin verdiği* hem de *neyi teşvik ettiği* ile kültürü şekillendirir. Örneğin:

- “Burada herkesin fikri değerlidir, stajyer dahil.” diyen ama toplantıda yalnızca senior’ları konuşturan bir lider, fiilen farklı bir mesaj verir.
- Hata sonrası, “Kim yaptı?” yerine, “Bunu sistemsel olarak nasıl önleriz?” sorusunu soran lider, blameless culture ve psychological safety’yi güçlendirir.
- “Bu olmaz.” cümlesi yerine, “Bunu deneseydik nerede takılırdık?” sorusunu tercih eden lider, hem eleştirel düşüncüyü hem de yenilikçiliği teşvik eder.

Sonuç olarak, yeni gelenlerin taze bakışını ciddiye almak, insanların emeğini görünür kılmak, işletme körlüğünü fark etmek ve “bu olmaz” önyargısını sorgulamak; ancak liderliğin bilinçli tutumlarıyla kalıcı şekilde içselleştirilebilir. Aksi hâlde, bireylerin iyi niyeti ve çabası olsun olmasın, sistem düzeyinde dönüşüm sınırlı kalır.

Sonuç

Bu dokümanda ele alınan kavramlar, teknoloji ekiplerinde liderlik ve sorumluluğu yalnızca unvan ve hiyerarşi üzerinden değil, **zihniyet**, **davranış** ve **sistem tasarımı** üzerinden düşünmeyi amaçlamaktadır. Ownership, accountability, responsibility, authority ve autonomy gibi temel kavramlar; tek tek tanımlandıklarında bile değerli olmakla birlikte, asıl anlamını birbirleriyle olan ilişkileri içinde bulur. Authority olmadan accountability talep etmek adil değildir; alignment olmadan autonomy vermek sahipsizlik üretir; due diligence ve stewardship olmadan da kısa vadeli teslimatlar uzun vadede sistemin çürümesine yol açar.

Bu çerçevede:

- **Bireyden** beklenen, kendi işine ve etki alanına sahip çıkan, gerekli özeni (due diligence) gösteren, gerektiğinde yardım isteyen ve sistemin bütününe gözetmeye çalışan bir profesyonellik tavrıdır.
- **Liderlikten** beklenen, rol ve beklentileri netleştiren, authority ile accountability'yi dengeli dağıtan, psychological safety ve blameless culture'ı bilinçli olarak inşa eden, yeni gelenlerin taze gözünü ciddiye alan ve insanların emeğini görünür kılan bir tasarım ve koruma rolüdür.
- **Organizasyondan** beklenen ise, DoR/DoD, RACI, rotasyon, review kültürü, dokümantasyon, onboarding gibi mekanizmaları kişisel iyi niyete bırakmadan, sistematik ve sürdürülebilir şekilde kurgulamaktır.

Sağlıklı bir teknoloji organizasyonu; hatayı cezalandırmak yerine öğrenmeyi teşvik eden, “bu olmaz” refleksi yerine merak ve sorgulamayı besleyen, sadece çıkan sonuca değil harcanan emeğe de saygı duyan bir kültür üzerine inşa edilir. Bu kültürde yeni gelenler işletme körlüğünü kıran birer ayna, deneyimliler ise yalnızca “daha fazla bilen” değil, aynı zamanda *daha fazla sorumluluk alan* ve alan açan kişiler hâline gelir.

Sonuç olarak, burada tanımlanan kavramlar birer soyut yönetim jargonu değil, günlük hayatta gözlemlenebilen şu sorularla somutlaştırılmalıdır:

- Bu işin gerçekten tekil bir *accountable* kişisi var mı?
- Bu sorumluluğu taşıyan kişinin elinde yeterli *authority* var mı?
- Ekipte insanlar soru sorarken ve hata paylaşıırken kendini güvende hissediyor mu?
- DoR ve DoD, herkesin aynı “bitti” tanımına bakmasını sağlıyor mu?
- Yeni gelenlerin eleştirileri ve soruları, sistemin kendini güncellemesine katkı sağlıyor mu?

Bu sorulara tutarlı ve dürüst cevaplar verilebildiği ölçüde; teknoloji ekipleri sadece “iş yapan” değil, öğrenen, kendini yenileyen ve insanlara adil davranan yaşayan sistemler hâline gelebilir.